МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій

Кафедра оптоелектроніки та інформаційних технологій

Звіт

До лабораторної роботи №1

Варіант 13

На тему "Основи функціонального програмування у Python"

Виконала:

Студентка групи ФЕП-21

Сокирка А. І.

Перевірив:

Ас. Остріков Д. А.

Львів 2026

Мета роботи:

- зрозуміти, що таке чиста функція і референтна прозорість;
- навчитися виявляти та ізолювати побічні ефекти;
- переписати імперативний фрагмент у функціональному стилі;
- застосувати typing.Callable для параметризації обчислень.

Теоретичні відомості:

- Чиста функція: детермінована (однаковий вхід → однаковий вихід), без змін глобального стану/вхідних даних, без I/O усередині.
- Референтна прозорість: вираз можна замінити його значенням без зміни поведінки програми.
- Побічні ефекти: I/O (print, файл, мережа), мутація вхідних структур, використання/зміна глобалів, залежність від time, random, os.environ тощо.
- Підхід: відокремити чисте ядро (перетворення даних) від оболонки ефектів (читання/запис).

Хід роботи:

Під час виконання лабораторної роботи було розроблено програму для обробки даних про страхові поліси. Для кожного поліса зберігаються ідентифікатор, базова страхова премія, рівень ризику, стаж клієнта та ознака активності.

У модулі core.py реалізовано функції для отримання базової премії, розрахунку премії з урахуванням ризику, застосування знижки за стаж і визначення остаточної вартості поліса. Для створення обробника використано make_processor, яка приймає функції відбору полісів, обчислення коефіцієнта ризику та розрахунку знижки.

Під час обробки програма включає до результату лише поліси, які відповідають переданій умові відбору. Для кожного з них формується новий словник з полем final_premium, тому початкові дані не змінюються. Наприкінці обчислюються кількість опрацьованих полісів і сума страхових премій.

У модулі app.py задано конкретні правила: коефіцієнт ризику дорівнює 1 + 0,5 × risk; для стажу від 5 років застосовується знижка 10 %, а від 10 років — 20 %. Неактивні поліси не включаються до підсумкового звіту.

Для перевірки програми створено тести pytest. Вони перевіряють однаковість результатів за повторного виклику, відсутність зміни вхідного списку,

правильність роботи переданих функцій, пропуск неактивних полісів, композицію функцій і створення функції-множника.

1. Файл core.py

```python
from collections.abc import Callable
from typing import TypedDict, TypeVar


class Policy(TypedDict):  10 usages
    id: int
    base_premium: float | int
    risk: float | int
    experience_years: int
    active: bool


class ProcessedPolicy(Policy):  3 usages
    final_premium: float | int


class PolicyResult(TypedDict):  3 usages
    count: int
    revenue: float | int
    policies: list[ProcessedPolicy]


RiskFn = Callable[[float], float]
DiscountFn = Callable[[float, int], float]
FilterFn = Callable[[Policy], bool]
```

2. Файл app.py

```python
from core import Policy, make_processor, render_report


def risk_coefficient(risk: float | int) -> float | int :  1 usage
    return 1.0 + risk * 0.5


def discount_policy(  1 usage
    premium: float | int,
    experience_years: int,
) -> float | int :
    if experience_years >= 10:
        discount = 0.20
    elif experience_years >= 5:
        discount = 0.10
    else:
        discount = 0.0

    return premium * (1.0 - discount)


def accept_policy(policy: Policy) -> bool:  1 usage
    return policy["active"]


def run() -> None:  1 usage
    policies: list[Policy] = [
        {
```

3. Файл test_core.py

```python
from copy import deepcopy

from core import (
    Policy,
    compose,
    make_multiplier,
    make_processor,
)


def sample_policies() -> list[Policy]:  4 usages
    return [
        {
            "id": 1,
            "base_premium": 1000.0,
            "risk": 1.0,
            "experience_years": 5,
            "active": True,
        },
        {
            "id": 2,
            "base_premium": 1500.0,
            "risk": 0.5,
            "experience_years": 12,
            "active": True,
        },
        {
            "id": 3,
```

4. Запуск програми

```
(.venv) PS C:\Users\4myryk\Desktop\Лаби сем 3\ФП\lab01> python app.py
Insurance policy report
Policy 1: 1350.00 UAH
Policy 2: 1500.00 UAH
Active policies processed: 2
Total premiums: 2850.00 UAH
```

5. Запуск тестів

```
(.venv) PS C:\Users\4myryk\Desktop\Лаби сем 3\ФП\lab01> python -m pytest -v
============================================================ test session starts
platform win32 -- Python 3.14.7, pytest-9.2.0, pluggy-1.7.0 -- C:\Users\4myryk\De
cachedir: .pytest_cache
rootdir: C:\Users\4myryk\Desktop\Лаби сем 3\ФП\lab01
plugins: platformdirs-4.14.0
collected 6 items

tests/test_core.py::test_referential_transparency PASSED
tests/test_core.py::test_no_mutation PASSED
tests/test_core.py::test_callable_policies PASSED
tests/test_core.py::test_inactive_policies_are_skipped PASSED
tests/test_core.py::test_compose PASSED
tests/test_core.py::test_make_multiplier PASSED
```

Висновок: Під час виконання лабораторної роботи було розроблено програму для обробки страхових полісів із використанням принципів функціонального програмування в Python. Обчислювальну логіку відокремлено від введення та виведення даних, а основні операції реалізовано у вигляді функцій, результат яких залежить від переданих аргументів.

За допомогою Callable реалізовано передавання правил відбору полісів, обчислення ризику та застосування знижок як функцій-аргументів. Програма створює нові записи з остаточною страховою премією, не змінюючи початковий список. Автоматизовані тести дають змогу перевірити стабільність результатів і коректність роботи програми. Отже, мету лабораторної роботи досягнуто.